

# Fine-Tuning Evaluation Metrics

*Perplexity, downstream benchmarks, LLM-as-judge, and human evaluation — a complete measurement toolkit*

**10 min**

Duration

**B4 – Analyse**

Bloom's

**L2 – Intermediate**

Level

**AI-FT-IN-TH-000017**

ID

# The 4 Evaluation Layers

## Layer 1: Perplexity

Measures model confidence on a held-out test set. Lower = better. Quick check during training. Does NOT correlate well with human preference.

## Layer 2: Benchmarks

MMLU (knowledge), HumanEval (code), MT-Bench (instruction following), TruthfulQA (factuality). Automated — fast and reproducible.

## Layer 3: LLM-as-Judge

Use GPT-4/Claude to score responses on a 1–5 rubric. Cheap, scalable, correlates well with human preference. Risks: judge biases.

## Layer 4: Human Evaluation

A/B comparisons by human raters. Most trustworthy but expensive (\$0.50–5.00/example). Required for final production decisions.

# Key Benchmark Suite for Fine-Tuned LLMs

| Benchmark  | Task Type                        | Metric                  | Fine-Tune Relevant?   |
|------------|----------------------------------|-------------------------|-----------------------|
| MMLU       | 57-subject knowledge test        | 5-shot accuracy         | ✔ Knowledge retention |
| HumanEval  | Python code generation           | pass@1 (exec. accuracy) | ✔ Code fine-tuning    |
| MT-Bench   | Multi-turn instruction following | GPT-4 score 1–10        | ✔ Chat fine-tuning    |
| TruthfulQA | Factual accuracy under pressure  | % truthful answers      | ✔ Hallucination check |
| AlpacaEval | Open-ended instruction quality   | Win rate vs GPT-4       | ✔ General fine-tuning |
| GSM8K      | Grade-school math reasoning      | CoT accuracy            | ✔ Reasoning tasks     |

# LLM-as-Judge Setup — GPT-4 Evaluation Prompt

```
import openai
client = openai.OpenAI()

JUDGE_PROMPT = """You are an expert AI evaluator. Rate the following
response to the instruction.

Instruction: {instruction}
Response: {response}

Rate on these dimensions (1-5 each):
1. Correctness: Is the response factually accurate?
2. Helpfulness: Does it fully address the instruction?
3. Clarity: Is it well-written and easy to understand?
4. Conciseness: Is it appropriately concise (not padded or truncated)?

Return ONLY valid JSON: {"correctness": N, "helpfulness": N, "clarity":
N, "conciseness": N, "overall": N}"""

def llm_judge(instruction, response):
    result = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": JUDGE_PROMPT.format(
            instruction=instruction, response=response)}],
        response_format={"type": "json_object"}
    )
    return json.loads(result.choices[0].message.content)

# Evaluate fine-tuned model vs base model
for ex in eval_set[:100]:
    ft_scores = llm_judge(ex["instruction"],
                           ft_model_output(ex["instruction"]))
    base_scores = llm_judge(ex["instruction"],
                             base_model_output(ex["instruction"]))
```

Key bias to watch: GPT-4 as judge has verbosity bias (prefers longer answers) and self-enhancement bias (prefers GPT-4-like outputs). Use multiple judge models.

## KEY REFERENCES

- Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023.
- Hendrycks, D. et al. (2020). Measuring Massive Multitask Language Understanding (MMLU). ICLR 2021.
- Chen, M. et al. (2021). Evaluating Large Language Models Trained on Code (HumanEval). arXiv.



**Next: Module 7 Assessment + Mini Project**